Foundations and Emerging Trends in AI for IoT

Armin MAZINANI

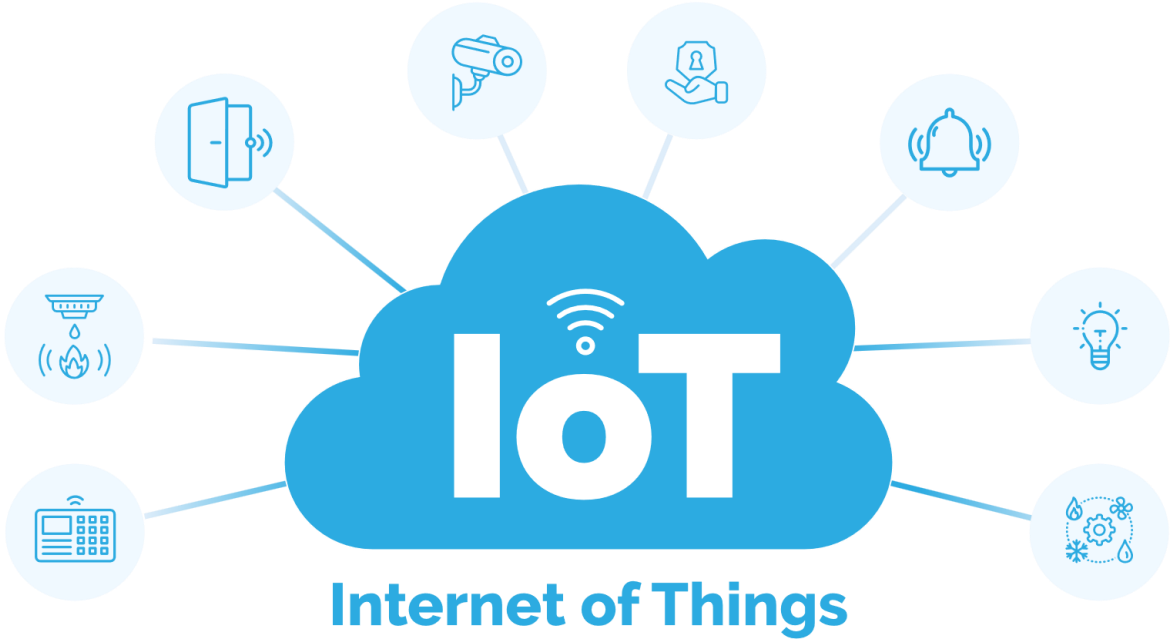
Internet of Things Lab, University of Parma
https://iotlab.unipr.it/

UNIVERSITÀ DI PARMA

IoT Lab

What is IoT?

- Network of connected devices
- Sensors and actuators
- Data collection and communication
- Cloud and edge integration



Motivation: Why AI for IoT?

- **Explosion of IoT devices**
  The rapid growth of connected sensors and smart devices is creating highly distributed and large-scale systems across industries (e.g., healthcare, industry, smart cities, etc.)
- **Massive data generation**
  Devices continuously generate large volumes of heterogeneous data (time-series, signals, images), making traditional processing approaches inefficient
- **Need for intelligent systems**
  To extract meaningful insights from *raw* data, AI techniques are required to enable automatic pattern recognition, prediction, and decision-making
- **Real-time decision-making use cases**
  Many IoT applications (e.g., industrial monitoring, autonomous systems, healthcare) require immediate responses with low latency and high reliability

Overview of AI in IoT

AI for IoT is the integration of **Artificial Intelligence (AI)** technologies (e.g., Machine Learning, ML, and Deep Learning, DL) into **IoT** systems to enable **intelligent decision-making** from the vast amounts of data generated by sensors and connected devices

Key Ideas

- IoT devices generate **continuous streams of data** (e.g., temperature, motion, video, or vibrations)
- AI processes this data to **detect patterns**, **predict outcomes**, or **make real-time decisions** without human intervention

Traditional IoT Architecture

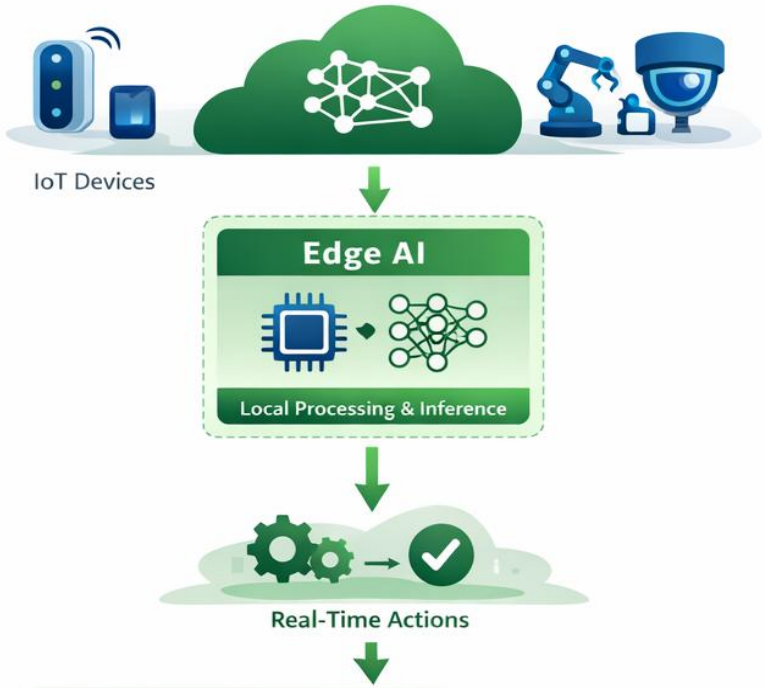
Centralized Processing Model



- High Cloud Dependency
- Higher Latency
- More Bandwidth Usage

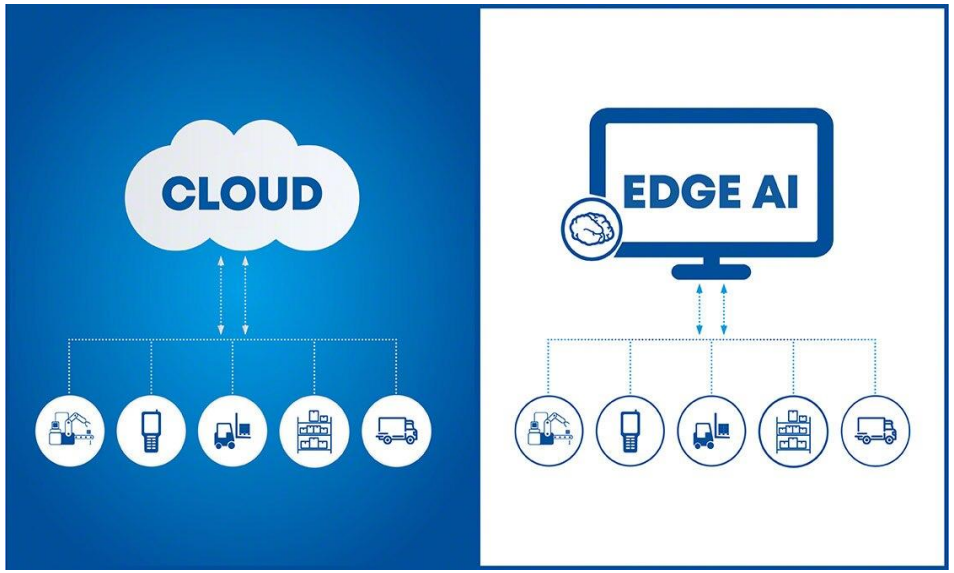
Intelligent Edge AI

Decentralized Processing Model



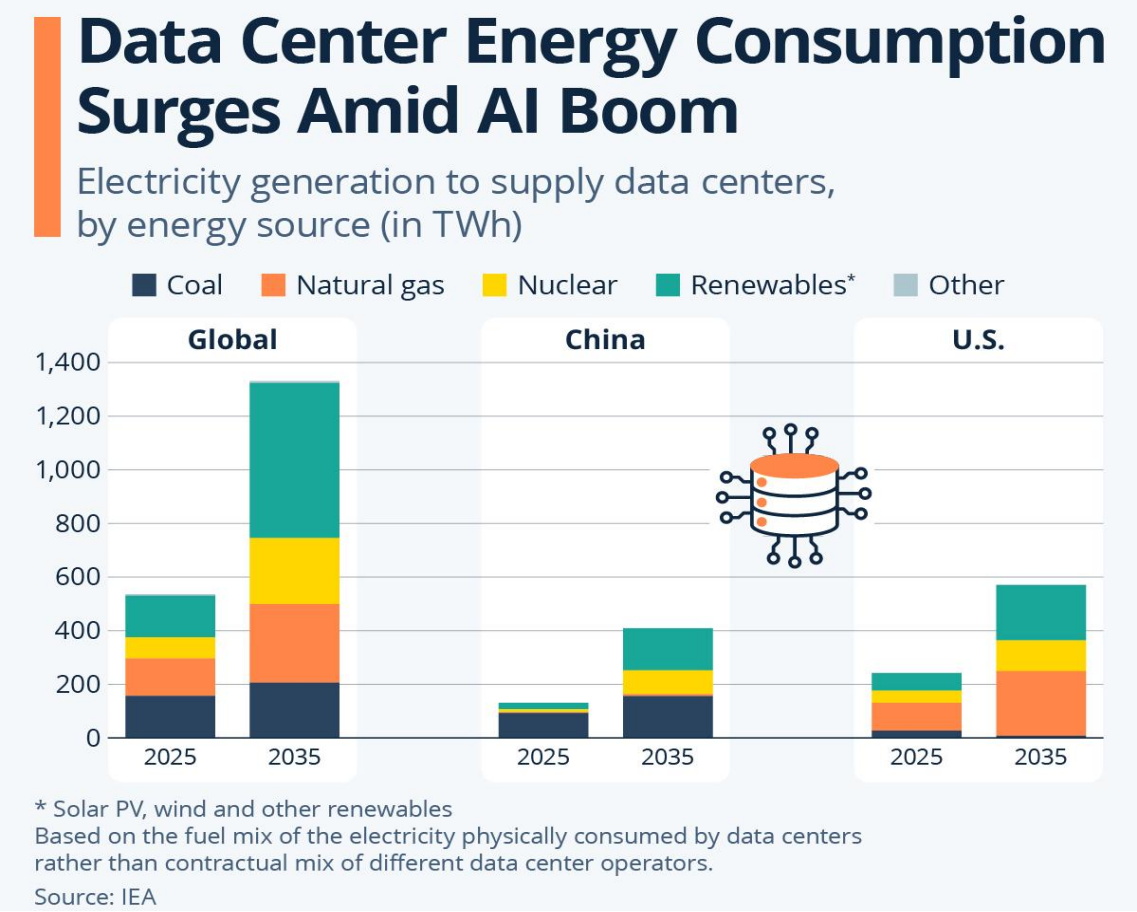
- On-Device Processing
- Faster Decisions
- Reduced Data Transmission

Comparison between Edge AI and Cloud AI



Feature	Edge AI	Cloud AI
Location of computation	On-device (microcontrollers, Raspberry Pi, gateways)	Remote servers / cloud
Latency	Very low, real-time decisions possible	Higher latency due to network delays
Bandwidth usage	Minimal (data processed locally)	High (*raw* data sent to cloud)
Privacy	Better (sensitive data stays on device)	Privacy risk if sending *raw* data to cloud
Power and resource needs	Limited resources → small/lightweight models	High compute power available

Global electricity demand from data centers is expected to more than double in the next five years, largely driven by the widespread uptake of AI



AI, ML & DL

- **AI**

Broad field enabling machines to perform intelligent tasks

Includes reasoning, perception, and decision-making

- **ML**

Subfield of AI that learns patterns from data

Requires feature extraction + training from examples

Examples: Support Vector Machine (SVM), Random Forest, *k*-Nearest Neighbors (*k*-NN)

- **DL**

Subfield of ML based on Neural Networks (NNs)

Automatically learns features from *raw* data

Examples: Convolutional NNs (CNNs), Recurrent NNs (RNNs), Long Short-Term Memory (LSTM), Transformers

Why large models are hard to deploy

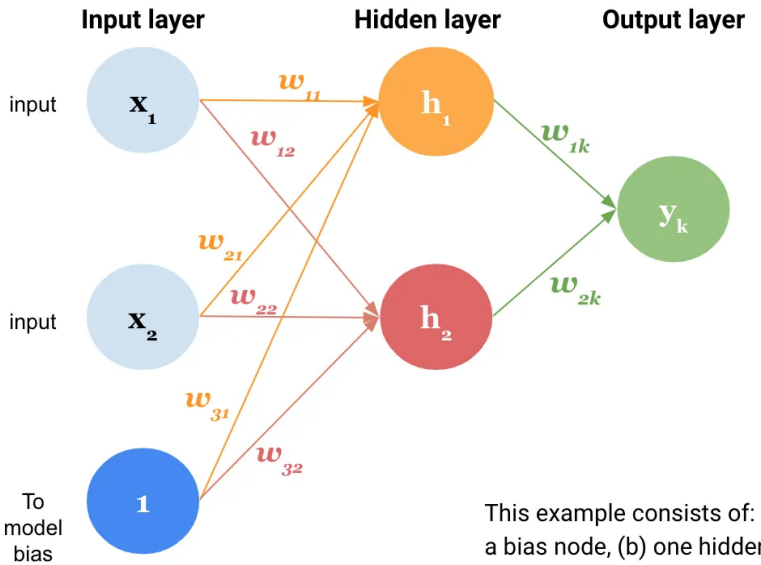
Large memory gap between billion-parameter LLMs and lightweight LLaMA-style models → Model size and numerical precision strongly impact feasibility for deployment on constrained hardware

Model	Parameters	Weight Precision	Model Size (weights)	Inference Profile
LLaMA-2	7B	FP32	≈ 28 GB	Memory-heavy inference
llama-lite	134M	4-bit quantized	≈ 85 MB	Lightweight / edge-friendly

Why DL for IoT?

- IoT generates high-dimensional time-series and sensor data
- Traditional ML struggles with complex patterns
- DL enables different operations
  - Automatic feature extraction
  - Better performance on *raw* signals
  - Handling nonlinear patterns
- Suitable for
  - Fault detection
  - Predictive maintenance
  - Activity recognition
  - Cybersecurity

MultiLayer Perceptron (MLP)



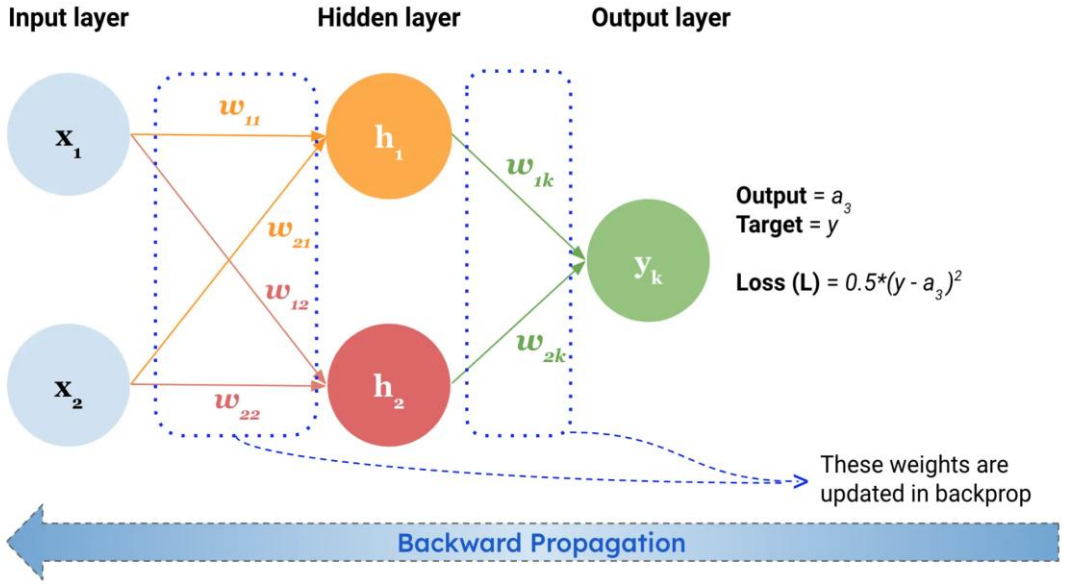
x_1 , x_2 : *input data features*

w_{ij} : *weights of the network*

h_1 , h_2 : *nodes in the hidden layer*

y_k : *output variable*

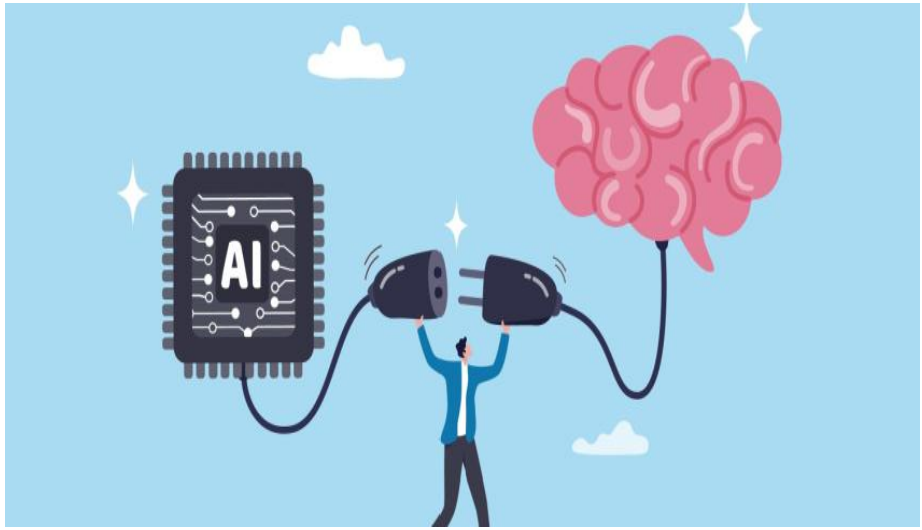
This example consists of: (a) an input layer with two input nodes and a bias node, (b) one hidden layer with two neurons, and (c) an output layer with one neuron



- In stochastic gradient descent, weights (w_{ij}) are updated as below:
  $w_{ij} := w_{ij} + \Delta w_{ij}$
  $\Delta w_{ij} = - \eta * \partial L(w_{ij}) / \partial w_{ij}$,
  where η is the learning rate and $\partial L(w_{ij}) / \partial w_{ij}$ is the gradient of loss w.r.t the model weights
- Intermediate variables calculated during forward prop ($z_1, z_2, z_3, a_1, a_2, a_3$) are used for gradient calculation $\partial L(w_{ij}) / \partial w_{ij}$

AI Deployment Pipeline for IoT

1. Data collection (sensors)
2. Data preprocessing (filtering, normalization)
3. Model training (cloud or local)
4. Model optimization
   - Quantization
   - Pruning
   - Knowledge distillation
5. Deployment
   - Edge device or microcontroller
6. Inference in *real-time*

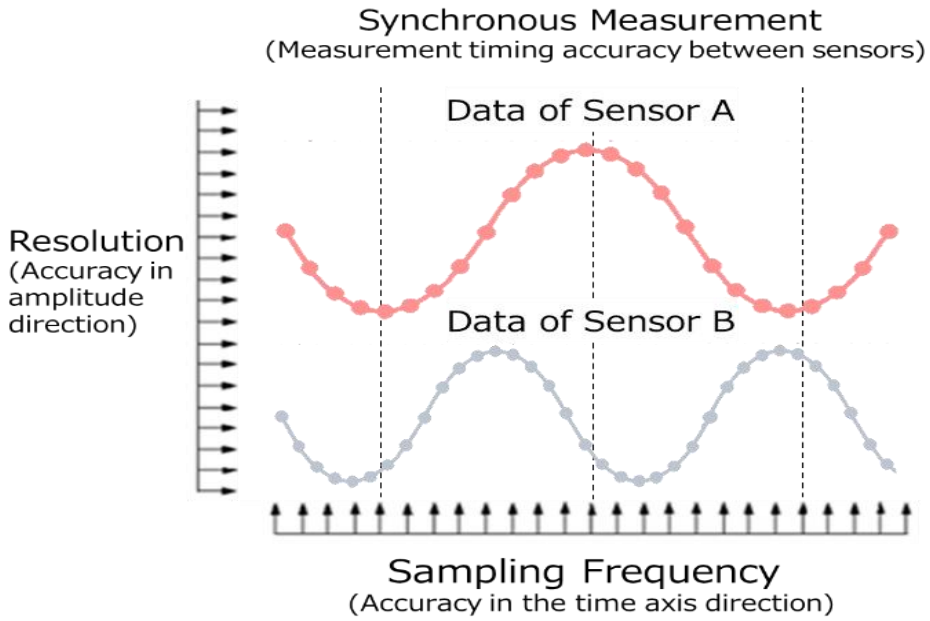


Data Collection (Sensors)

First stage where *raw* data are acquired from physical devices such as sensors, actuators, or embedded systems

Examples include temperature, vibration, current, voltage, audio, or motion signals

The quality and sampling rate of sensors directly affect the performance of the AI model.



Data Preprocessing

Raw sensor data are often noisy, incomplete, or inconsistent

Data are cleaned and transformed to make it suitable for ML

Filtering: Removes noise and unwanted signal components (e.g., low-pass filters, moving average)

Normalization/Scaling: Adjusts values to a common range (e.g., 0–1 or *z*-score) to stabilize training

Segmentation: Splits continuous signals into windows or frames

Handling missing data: Imputation or removal of corrupted samples

Model Optimization

After training, the model is optimized to reduce size, computation, and memory usage—especially important for edge deployment

Quantization: Reduces numerical precision (e.g., float32 → int8) to make models lighter and faster

Pruning: Removes unnecessary weights or neurons to reduce complexity

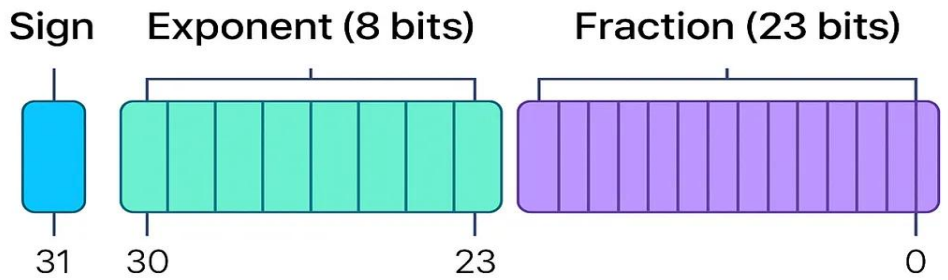
Knowledge Distillation: A smaller “student” model learns from a larger “teacher” model, preserving performance with fewer resources

Quantization

Not all numbers are created equal in computing. There are different **precision types** that define how a number is stored in binary. The choice of precision has a direct impact on a model's memory footprint and computation speed.

- **FP32 (32-bit Floating Point)**
  - FP32 uses 32 bits (4 bytes) per number

FP32 was the standard for training neural networks for years because it offers high precision and range to avoid numerical issues. However, **FP32 consumes a lot of memory and compute. Every weight, activation, and gradient is 4 bytes**.



Quantization

INT8 have no concept of “fractional” bits or exponents.

An 8-bit signed integer can represent values roughly from -128 to +127 (or 0 to 255 if unsigned).

INT8 quantization involves mapping real-valued weights/activations to an 8-bit integer scale.

The primary appeal is efficiency: an INT8 number is 4x smaller than an FP32 value, so models shrink to a quarter of their size.



Quantization

- **Model compression:** smaller weights/activations (e.g., FP32 → INT8) ⇒ lower storage + lower RAM footprint
- **Faster inference :** low-precision / integer arithmetic ⇒ reduced latency (often significant on supported CPUs/accelerators)
- **Lower energy / power:** fewer bits moved + cheaper ops ⇒ reduced energy consumption and better suitability for edge/embedded deployment

Quantization

- INT8 quantization equation

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

$$\hat{x} = s \cdot (q - z)$$

x = real (float32) value

q = quantized integer (e.g., INT8 → −128 to 127)

s = scale (float)

z = zero-point (integer offset)

- How scale and zero-point are computed

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$$

$$z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right)$$

Quantization

- $x_{\min} = -1.0, x_{\max} = 1.0$
- INT8 $\rightarrow q \in [-128, 127]$

Step 1: scale

$$s = \frac{2}{255} \approx 0.00784$$

Step 2: zero-point

$$z = \text{round}(-128 - (-1/0.00784)) \approx 0$$

So:

$$q = \text{round}(x/0.00784)$$

Post-Training Quantization (PTQ)

PTQ applied **after the model is fully trained**

- Converts trained weights from high precision (e.g., FP32) to lower precision (e.g., INT8)
- No retraining required (or only a small calibration step)
- Uses a small calibration dataset to estimate activation ranges
- Fast and simple to apply

Advantages

- Very easy to implement
- No need to change training pipeline
- Reduces memory and improves inference speed

Quantization-Aware Training (QAT)

QAT is **simulated during training**

Model learns to adapt to low-precision constraints

Advantages

- Higher accuracy compared to PTQ
- Better robustness to INT8/FP16 deployment
- More suitable for critical applications (IoT, medical, industrial AI)

Limitations

- Requires retraining the model
- Higher computational cost during training
- More complex pipeline

Knowledge Distillation (KD)

- **Model compression**: train a small student to mimic a large teacher
- Student learns from teacher **soft predictions** (richer than hard labels)
- Goal: **preserve performance** while **reducing compute** → easier deployment on constrained devices

Teacher–Student Framework

- Teacher: large, accurate, but computationally expensive
- Student: smaller, faster, lightweight
- Training target: match teacher's **probability distribution**, not only the ground-truth class

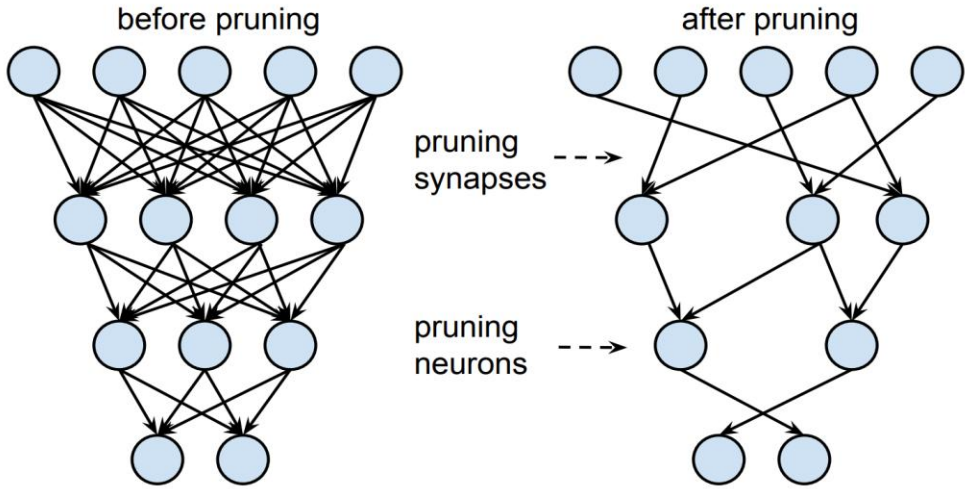
Knowledge Distillation (KD)

- Use **Softmax with temperature (T)** to “soften” teacher outputs
- Higher **T** reveals class similarities (“dark knowledge”) in the teacher distribution
- Student is trained to reproduce these softened outputs (then use **T=1** at inference)

KD = accuracy-aware compression: transfer teacher behavior → smaller model with similar performance at lower cost

NN Pruning

- Pruning is removing parameters from an existing neural network to **reduce size/compute** while trying to keep accuracy.
- **Weight pruning:** set individual weights to zero → sparse model.
  - Removing a weight is essentially setting it to zero; prune weights already close to 0 (low magnitude) to reduce impact.



Tutorials:

KD tutorial

https://colab.research.google.com/drive/1aztIQvrsDdnAv8M75wNqA30D3Xi-JHdt?usp=sharing

Quantization and pruning tutorial

https://colab.research.google.com/drive/1A7QHajTdtkuBEQY3yZzGKj1XCRWMRf2P?usp=sharing



KD



Quantization

Federated Learning (FL)

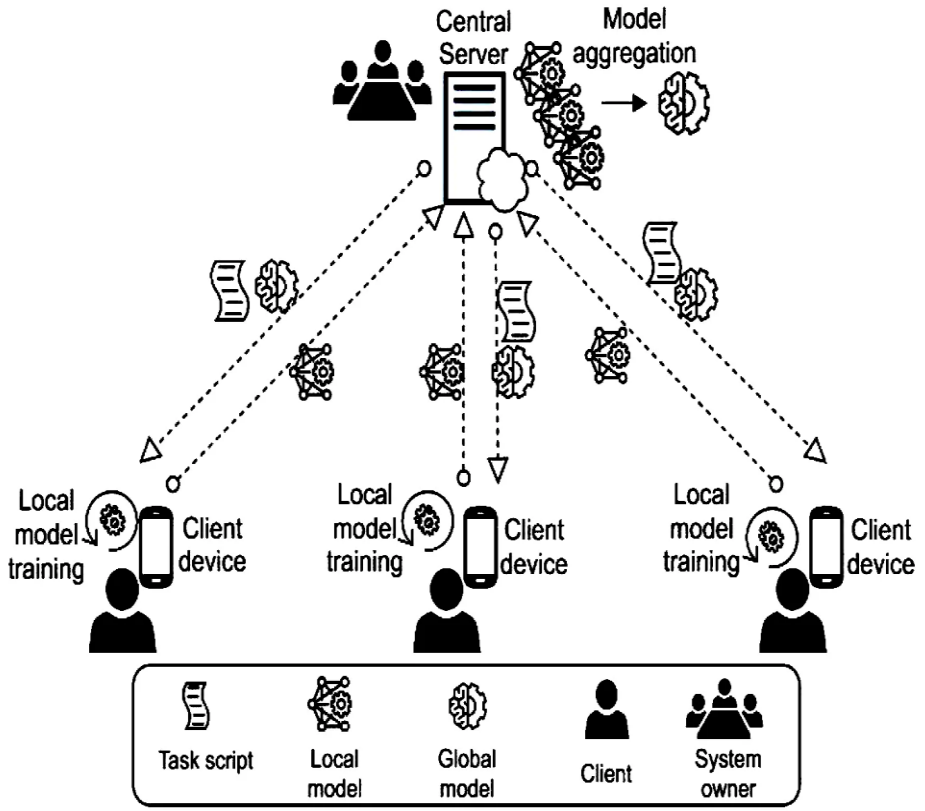
Federated Learning enables multiple clients to collaboratively train a shared model while keeping training data local.

- Local training on edge devices
- Only model updates are shared
- Central server aggregates updates

“Move the model to the data, not the data to the model”

Federated Learning (FL)

- Server initializes global model
- Model is sent to selected clients
- Each client trains locally
- Clients send updates (weights / gradients)
- Server aggregates (e.g., averaging)
- Repeat until convergence



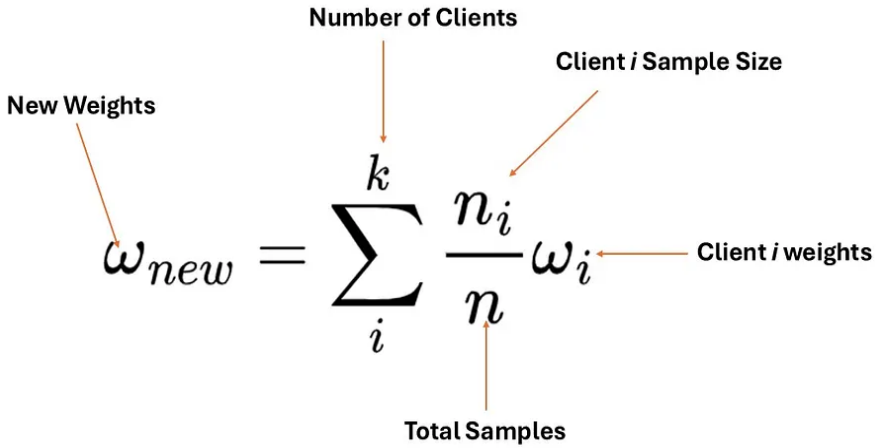
FedAvg

Three clients were involved in a FL round and they each have sent their weights to the server say:

w1=[1.1,0.9] from 100 samples

w2=[1.3,1.0] from 300 samples

w3=[1.2,0.95] from 100 samples



$$\omega_{new} = \frac{100 \cdot \omega_1 + 300 \cdot \omega_2 + 100 \cdot \omega_3}{100 + 300 + 100}$$

$$\omega_{new} = \frac{100 \cdot (1.1, 0.9) + 300 \cdot (1.3, 1.0) + 100 \cdot (1.2, 0.95)}{500}$$

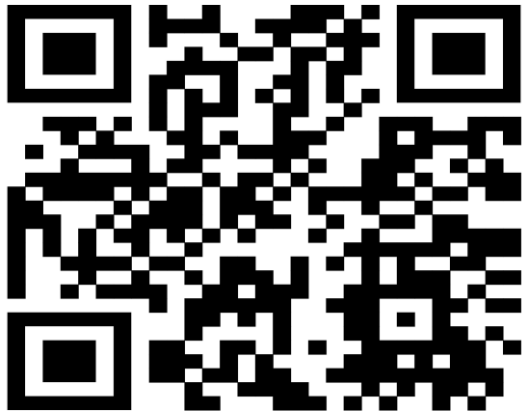
$$\omega_{new} = \frac{(110, 90) + (390, 300) + (120, 95)}{500}$$

$$\omega_{new} = (1.24, 0.97)$$

Tutorials:

FL tutorial:

https://colab.research.google.com/drive/1EPSkcJMhlL3vH_HlZOTF7vlOJbYF2OBe?usp=sharing



FL